Getting Started
This is ZeroTier
The Protocol
ZeroTier Security
Rules Engine
Client Configuration
What is a Network Controller?
Root Server IP Whitelist
Guides
OS / Platform Notes
FAQ & Troubleshooting
Glossary

This is ZeroTier > The Protocol

The Protocol

Overview
Origin and Design Philosophy
Network Hypervisor
The ZeroTier Peer to Peer Network (VL1)
Network Topology and Peer Discovery
Addressing
Cryptography
Trusted Paths for Fast Local SDN
Ethernet Virtualization Layer (VL2)
Network Identifiers and Controllers
Certificates and Other Credentials
Multicast, ARP, NDP, and

ZeroTier is a smart programmable Ethernet switch for planet Earth. It allows all networked devices, VMs, containers, and applications to communicate as if they all reside in the same physical data center or cloud region.

Overview

ZeroTier is a distributed network hypervisor built atop a cryptographically secure global peer to peer network. It provides advanced network virtualization and management capabilities on par with an enterprise SDN switch, but across both local and wide area networks and connecting almost any kind of app or device.

This is accomplished by combining a cryptographically addressed and secure peer to peer network (termed VL1) with an Ethernet emulation layer somewhat similar to VXLAN (termed VL2). Our VL2 Ethernet virtualization layer includes advanced enterprise SDN features like fine grained access control rules for network micro-segmentation and security monitoring.

All ZeroTier traffic is encrypted end-to-end using secret keys that only you control. Most traffic flows peer to peer, though we offer free (but slow) relaying for users who cannot establish peer to peer connections.

Everything in the ZeroTier world is controlled by two types of identifier: 40-bit/10-digit *ZeroTier addresses* and 64-bit/16-digit *network IDs*. These identifiers are easily distinguished by their length. A ZeroTier address identifies a node or "device" (laptop, phone, server, VM, app, etc.) while a network ID identifies a virtual Ethernet network that can be joined by devices.

ZeroTier addresses can be thought of as port numbers on an enormous planet-wide enterprise Ethernet smart switch supporting VLANs. Network IDs are VLAN IDs to which these ports may be assigned. A single port can be assigned to more than one VLAN.

A ZeroTier address looks like `8056c2e21c` and a network ID looks like `8056c2e21c000001`. Network IDs are composed of the ZeroTier address of that network's primary controller and an arbitrary 24-bit ID that identifies the network on this controller. Network controllers are roughly analogous to SDN controllers in SDN protocols like OpenFlow, though as with the analogy between VXLAN and VL2 this should not be read to imply that the protocols or design are the same. You can use our convenient and inexpensive SaaS hosted controllers at my.zerotier.com or run your own controller if you don't mind messing around with JSON configuration files or writing scripts to do so.

> ⓘ INFO
>
> Visit ZeroTier's site for more information and pre-built binary packages. Apps for Android and iOS are available for free in the Google Play and Apple app stores.

Origin and Design Philosophy

The goals and design principles of ZeroTier are inspired by among other things the original Google BeyondCorp paper and the Jericho Forum with its notion of "deperimeterization."

Network Hypervisor

The ZeroTier network hypervisor (currently found in the node/ subfolder of the ZeroTierOne git repository) is a self-contained network virtualization engine that implements an Ethernet virtualization layer similar to VXLAN on top of a global encrypted peer to peer network.

The ZeroTier protocol is original, though aspects of it are similar to VXLAN and IPSec. It has two conceptually separate but closely coupled layers in the OSI model sense: **VL1** and **VL2**. VL1 is the underlying peer to peer transport layer, the "virtual wire," while VL2 is an emulated Ethernet layer that provides operating systems and apps with a familiar communication medium.

The ZeroTier Peer to Peer Network (VL1)

A global data center requires a global wire closet.

In conventional networks L1 (OSI layer 1) refers to the actual CAT5/CAT6 cables or wireless radio channels over which data is carried and the physical transceiver chips that modulate and demodulate it. VL1 is a peer to peer network that does the same thing by using encryption, authentication, and a lot of networking tricks to create virtual wires on a dynamic as-needed basis.

Network Topology and Peer Discovery

VL1 is designed to be zero-configuration. A user can start a new ZeroTier node without having to write configuration files or provide the IP addresses of other nodes. It's also designed to be fast. Any two devices in the world should be able to locate each other and communicate almost instantly.

To achieve this VL1 is organized like DNS. At the base of the network is a collection of always-present **root servers** whose role is similar to that of DNS root name servers. Roots run the same software as regular endpoints but reside at fast stable locations on the network and are designated as such by a **world definition**. World definitions come in two forms: the **planet** and one or more **moons**. The protocol includes a secure mechanism allowing world definitions to be updated in-band if root servers' IP addresses or ZeroTier addresses change.

There is only one planet. Earth's root servers are operated by ZeroTier, Inc. as a free service. There are currently four root servers distributed across the globe and multiple network providers. Almost everyone in the world has one within less than 100ms network latency from their location.

A node can "orbit" any number of moons. A moon is just a convenient way to add user-defined root servers to the pool. Users can create moons to reduce dependency on ZeroTier, Inc. infrastructure or to locate root servers closer for better performance. For on-premise SDN use a cluster of root servers can be located inside a building or data center so that ZeroTier can continue to operate normally if Internet connectivity is lost.

Nodes start with no direct links to one another, only upstream to roots (planet and moons). Every peer on VL1 possesses a globally unique 40-bit (10 hex digit) **ZeroTier address**, but unlike IP addresses these are opaque cryptographic identifiers that encode no routing information. To communicate peers first send packets "up" the tree, and as these packets traverse the network they trigger the opportunistic creation of direct links along the way. The tree is constantly trying to "collapse itself" to optimize itself to the pattern of traffic it is carrying.

Peer to peer connection setup goes like this:

1. A wants to send a packet to B, but since it has no direct path it sends it upstream to R (a root).
2. If R has a direct link to B, it forwards the packet there. Otherwise it sends the packet upstream until planetary roots are reached. Planetary roots know about all nodes, so eventually the packet will reach B if B is online.
3. R also sends a message called *rendezvous* to A containing hints about how it might reach B. Meanwhile the root that forwards the packet to B sends *rendezvous* informing B how it might reach A.
4. A and B get their *rendezvous* messages and attempt to send test messages to each other, possibly accomplishing hole punching of any NATs or stateful firewalls that happen to be in the way. If this works a direct link is established and packets no longer need to take the scenic route.

Since roots forward packets, A and B can reach each other instantly. A and B then begin attempting to make a direct peer to peer connection. If this succeeds it results in a faster lower latency link. We call this *transport triggered link provisioning* since it's the forwarding of the packet itself that triggers the peer to peer network to attempt direct connection.

VL1 never gives up. If a direct path can't be established, communication can continue through (slower) relaying. Direct connection attempts continue forever on a periodic basis. VL1 also has other features for establishing direct connectivity including LAN peer discovery, port prediction for traversal of symmetric IPv4 NATs, and explicit port mapping using uPnP and/or NAT-PMP if these are available on the local physical LAN.

A blog post from 2014 by ZeroTier's original author explains some of the reasoning behind VL1's design.

Addressing

Every node is uniquely identified on VL1 by a 40-bit (10 hex digit) **ZeroTier address**. This address is computed from the public portion of a public/private key pair. A node's address, public key, and private key together form its **identity**.

On devices running ZeroTier One the node identity is stored in `identity.public` and `identity.secret` in the service's home directory.

When ZeroTier starts for the first time it generates a new identity. It then attempts to advertise it upstream to the network. In the very unlikely event that the identity's 40-bit unique address is taken, it discards it and generates another.

Identities are claimed on a first come first serve basis and currently expire from planetary roots after 60 days of inactivity. If a long-dormant device returns it may re-claim its identity unless its address has been taken in the meantime (again, highly unlikely).

The address derivation algorithm used to compute addresses from public keys imposes a computational cost barrier against the intentional generation of a collision. Currently it would take approximately 10,000 CPU-years to do so (assuming e.g. a 3ghz Intel core). This is expensive but not impossible, but it's only the first line of defense. After generating a collision an attacker would then have to compromise all upstream nodes, network controllers, and anything else that has recently communicated with the target node and replace their cached identities.

ZeroTier addresses are, once advertised and claimed, a very secure method of unique identification.

When a node attempts to send a message to another node whose identity is not cached, it sends a *whois* query upstream to a root. Roots provide an authoritative identity cache.

Cryptography

If you don't know much about cryptography you can safely skip this section. **TL;DR: packets are end-to-end encrypted and can't be read by roots or anyone else, and we use modern 256-bit crypto in ways recommended by the professional cryptographers that created it.**

Asymmetric public key encryption is Curve25519/Ed25519, a 256-bit elliptic curve variant.

Every VL1 packet is encrypted end to end using (as of the current version) 256-bit Salsa20 and authenticated using the Poly1305 message authentication (MAC) algorithm. MAC is computed after encryption (encrypt-then-MAC) and the cipher/MAC composition used is identical to the NaCl reference implementation.

As of today we do not implement forward secrecy or other stateful cryptographic features in VL1. We don't do this for the sake of simplicity, reliability, and code footprint, and because frequently changing state makes features like clustering and fail-over much harder to implement. See our discussion on GitHub.

We may implement forward secrecy in the future. For those who want this level of security today, we recommend using other cryptographic protocols such as SSL or SSH over ZeroTier. These protocols typically implement forward secrecy, but using them over ZeroTier also provides the secondary benefit of defense in depth. Most cryptography is compromised not by a flaw in encryption but through bugs in the implementation. If you're using two secure transports, the odds of a critical bug being discovered in both at the same time is very low. The CPU overhead of double-encryption is not significant for most work loads.

> ⓘ SECURITY
>
> More information on ZeroTier's security practices

Trusted Paths for Fast Local SDN

To support the use of ZeroTier as a high performance SDN/NFV protocol over physically secure networks the protocol supports a feature called *trusted paths*. It is possible to configure all ZeroTier devices on a given network to skip encryption and authentication for traffic over a designated physical path. This can cut CPU use noticeably in high traffic scenarios but at the cost of losing virtually all transport security.

Trusted paths do not prevent communication with devices elsewhere, since traffic over other paths will be encrypted and authenticated normally.

We don't recommend the use of this feature unless you really need the performance and you know what you're doing. We also recommend thinking carefully before disabling transport security on a cloud private network. Larger cloud providers such as Amazon and Azure tend to provide good network segregation but many less costly providers offer private networks that are "party lines" and are not much more secure than the open Internet.

Ethernet Virtualization Layer (VL2)

VL2 is a VXLAN-like network virtualization protocol with SDN management features. It implements secure VLAN boundaries, multicast, rules, capability based security, and certificate based access control.

VL2 is built atop and carried by VL1, and in so doing it inherits VL1's encryption and endpoint authentication and can use VL1 asymmetric keys to sign and verify credentials. VL1 also allows us to implement VL2 entirely free of concern for underlying physical network topology. Connectivity and routing efficiency issues are VL1 concerns. It's important to understand that there is no relationship between VL2 virtual networks and VL1 paths. Much like VLAN multiplexing on a wired LAN, two nodes that share multiple network memberships in common will still only have one VL1 path (virtual wire) between them.

Network Identifiers and Controllers

Each VL2 network (VLAN) is identified by a 64-bit (16 hex digit) **ZeroTier network ID** that contains the 40-bit ZeroTier address of the network's **controller** and a 24-bit number identifying the network on the controller.

```
Network ID: 8056c2e21c123456
            |         |
            |         Network number on controller
            |
            ZeroTier address of controller
```

When a node joins a network or requests a network configuration update, it sends a network config query message (via VL1) to the network's controller. The controller can then use the node's VL1 address to look it up on the network and send it the appropriate certificates, credentials, and configuration information. From the perspective of VL2 virtual networks, VL1 ZeroTier addresses can be thought of as port numbers on an enormous global-scale virtual switch.

A common misunderstanding is to conflate network controllers with root servers (planet and moons). Root servers are connection facilitators that operate at the VL1 level. Network controllers are configuration managers and certificate authorities that belong to VL2. Generally root servers don't join or control virtual networks and network controllers are not root servers, though it is possible to have a node do both.

Controller Security Considerations

Network controllers serve as certificate authorities for ZeroTier virtual networks. As such, their `identity.secret` files should be guarded closely and backed up securely. Compromise of a controller's secret key would allow an attacker to issue fraudulent network configurations or admit unauthorized members, while loss of the secret key results in loss of ability to control the network in any way or issue configuration updates and effectively renders the network unusable.

It is important that controllers' system clocks remain relatively accurate (to within 30-60 seconds) and that they are secure against remote tampering. Many cloud providers provide secure time sources either directly via the hypervisor or via NTP servers within their networks.

Certificates and Other Credentials

All credentials issued by network controllers to member nodes in a given network are signed by the controller's secret key to allow all network members to verify them. Credentials have timestamp fields populated by the controller, allowing relative comparison without the need to trust the node's local system clock.

Credentials are issued only to their owners and are then pushed peer to peer by nodes that wish to communicate with other nodes on the network. This allows networks to grow to enormous sizes without requiring nodes to cache large numbers of credentials or to constantly consult the controller.

Credential Types

- **Certificates of Membership**: a certificate that a node presents to obtain the right to communicate on a given network. Certificates of membership are accepted if they *agree*, meaning that the submitting member's certificate's timestamp differs from the recipient's certificate's timestamp by no more than the recipient certificate's maximum timestamp delta value. This creates a decentralized moving-window scheme for certificate expiration without requiring node clock synchronization or constant checking with the controller.
- **Revocations**: a revocation instantaneously revokes a given credential by setting a hard timestamp limit before which it will not be accepted. Revocations are rapidly propagated peer to peer among members of a network using a rumor mill algorithm, allowing a controller to revoke a member credential across the entire network even if its connection to some members is unreliable.
- **Capabilities**: a capability is a bundle of network rules that is signed by the controller and can be presented to other members of a network to grant the presenter elevated privileges within the framework of the network's base rule set. More on this in the section on rules.
- **Tags**: a tag is a key/value pair signed by the controller that is automatically presented by members to one another and can be matched on in base or capability network rules. Tags can be used to categorize members by role, department, classification, etc.
- **Certificates of Ownership**: these certify that a given network member owns something, such as an IP address. These are currently only used to lock down networks against IP address spoofing but could be used in the future to certify ownership of other network-level entities that can be matched in a filter.

Multicast, ARP, NDP, and Special Addressing Modes

ZeroTier networks support multicast via a simple publish/subscribe system.

When a node wishes to receive multicasts for a given multicast group, it advertises membership in this group to other members of the network with which it is communicating and to the network controller. When a node wishes to send a multicast it both consults its cache of recent advertisements and periodically solicits additional advertisements.

Broadcast (Ethernet *ff:ff:ff:ff:ff:ff*) is treated as a multicast group to which all members subscribe. It can be disabled at the network level to reduce traffic if it is not needed. IPv4 ARP receives special handling (see below) and will still work if normal broadcast is disabled.

Multicasts are propagated using simple sender-side replication. This places the full outbound bandwidth load for multicast on the sender and minimizes multicast latency. Network configurations contain a network-wide **multicast limit** configurable at the network controller. This specifies the maximum number of other nodes to which any node will send a multicast. If the number of known recipients in a given multicast group exceeds the multicast limit, the sender chooses a random subset.

There is no global limit on multicast recipients, but setting the multicast limit very high on very large networks could result in significant bandwidth overhead.

Special Handling of IPv4 ARP Broadcasts

IPv4 ARP is built on simple Ethernet broadcast and scales poorly on large or distributed networks. To improve ARP's scalability ZeroTier generates a unique multicast group for each IPv4 address detected on its system and then transparently intercepts ARP queries and sends them only to the correct group. This converts ARP into effectively a unicast or narrow multicast protocol (like IPv6 NDP) and allows IPv4 ARP to work reliably across wide area networks without excess bandwidth consumption. A similar strategy is implemented under the hood by a number of enterprise switches and WiFi routers designed for deployment on extremely large LANs. This ARP emulation mode is transparent to the OS and application layers, but it does mean that packet sniffers will not see all ARP queries on a virtual network the way they typically can on smaller wired LANs.

Multicast-Free IPv6 Addressing Modes

IPv6 uses a protocol called NDP in place of ARP. It is similar in role and design but uses narrow multicast in place of broadcast for superior scalability on large networks. This protocol nevertheless still imposes the latency of an additional multicast lookup whenever a new address is contacted. This can add hundreds of milliseconds over a wide area network, or more if latencies associated with pub/sub recipient lookup are significant.

IPv6 addresses are large enough to easily encode ZeroTier addresses. For faster operation and better scaling we've implemented several special IPv6 addressing modes that allow the local node to emulate NDP. These are ZeroTier's **rfc4193** and **6plane** IPv6 address assignment schemes. If these addressing schemes are enabled on a network, nodes locally intercept outbound NDP queries for matching addresses and then locally generate spoofed NDP replies.

Both modes dramatically reduce initial connection latency between network members. **6plane** additionally exploits NDP emulation to transparently assign an entire IPv6 /80 prefix to every node without requiring any node to possess additional routing table entries. This is designed for virtual machine and container hosts that wish to auto-assign IPv6 addresses to guests and is very useful on microservice architecture backplane networks.

Finally there is a security benefit to NDP emulation. ZeroTier addresses are cryptographically authenticated, and since Ethernet MAC addresses on networks are computed from ZeroTier addresses these are also secure. NDP emulated IPv6 addressing modes are therefore not vulnerable to NDP reply spoofing.

Normal non-NDP-emulated IPv6 addresses (including link-local addresses) can coexist with NDP-emulated addressing schemes. Any NDP queries that do not match NDP-emulated addresses are sent via normal multicast.

Ethernet Bridging

ZeroTier emulates a true Ethernet switch. This includes the ability to L2 bridge other Ethernet networks (wired LAN, WiFi, virtual backplanes, etc.) to virtual networks using conventional Ethernet bridging.

To act as a bridge a network member must be designated as such by the controller. This is for security reasons as normal network members are not permitted to send traffic from any origin other than their MAC address. Designated bridges also receive special treatment from the multicast algorithm, which more aggressively and directly queries them for group subscriptions and replicates all broadcast traffic and ARP requests to them. As a result bridge nodes experience a slightly higher amount of multicast bandwidth overhead.

Bridging has been tested extensively on Linux using the Linux kernel native bridge, which cleanly handles network MTU mismatch. There are third party reports of bridging working on other platforms. The details of setting up bridging, including how to selectively block traffic like DHCP that may not be wanted across the bridge, are beyond the scope of this manual.

> ⓘ GUIDE
>
> See our bridging tutorial

Public Networks

It is possible to disable access control on a ZeroTier network. A public network's members do not check certificates of membership, and new members to a public network are automatically marked as authorized by their host controller. It is not possible to de-authorize a member from a public network.

Rules on the other hand *are* enforced, so it's possible to implement a special purpose public network that only allows access to a few things or that only allows a restricted subset of traffic.

Public networks are useful for testing and for peer to peer "party lines" for gaming, chat, and other applications. **Participants in public networks are warned to pay special attention to security. If joining a public network be careful not to expose vulnerable services or accidentally share private files via open network shares or HTTP servers. Make sure your operating system, applications, and services are fully up to date.**

Ad-Hoc Networks

A special kind of public network called an ad-hoc network may be accessed by joining a network ID with the format:

```
ffSSSSEEEE000000
| |   |   |
| |   |   Reserved for future use, must be 0
| |   End of port range (hex)
| Start of port range (hex)
Reserved ZeroTier address prefix indicating a controller-less network
```

Ad-hoc networks are public (no access control) networks that have no network controller. Instead their configuration and other credentials are generated locally. Ad-hoc networks permit only IPv6 UDP and TCP unicast traffic (no multicast or broadcast) using 6plane format NDP-emulated IPv6 addresses. In addition an ad-hoc network ID encodes an IP port range. UDP packets and TCP SYN (connection open) packets are only allowed to destination ports within the encoded range.

For example `ff00160016000000` is an ad-hoc network allowing only SSH, while `ff0000ffff000000` is an ad-hoc network allowing any UDP or TCP port.

Keep in mind that these networks are public and anyone in the entire world can join them. Care must be taken to avoid exposing vulnerable services or sharing unwanted files or other resources.

> ⓘ GETTING STARTED
>
> Click here to create your network and start adding devices.

Tags: beginner

Copyright © 2025 ZeroTier, Inc. All rights reserved.